

SKILL SESSION-5

Task 1 : To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
vennela_07@VennelaPC:~$ mkdir single_quote_parser
cd single_quote_parser
vennela_07@VennelaPC:~/single_quote_parser$ pwd
/home/vennela_07/single_quote_parser
vennela_07@VennelaPC:~/single_quote_parser$ nano quote_parser.c
vennela_07@VennelaPC:~/single_quote_parser$ gcc quote_parser.c -o quote_parser
vennela_07@VennelaPC:~/single_quote_parser$ ./quote_parser
Enter a command: echo 'Hello World'

--- Parsing Result ---
Original Input : echo 'Hello World'
Stored String  : echo Hello World

--- Validation ---
Single quotes handled successfully.
Literal content preserved.
Variable expansion is ignored.
Parsing result is valid.
vennela_07@VennelaPC:~/single_quote_parser$ ./quote_parser
Enter a command: echo '$HOME'

--- Parsing Result ---
Original Input : echo '$HOME'
Stored String  : echo $HOME

--- Validation ---
Single quotes handled successfully.
Literal content preserved.
Variable expansion is ignored.
Parsing result is valid.
vennela_07@VennelaPC:~/single_quote_parser$ ./quote_parser
Enter a command: echo 'Hello      World'

--- Parsing Result ---
Original Input : echo 'Hello      World'
Stored String  : echo Hello      World

--- Validation ---
Single quotes handled successfully.
Literal content preserved.
Variable expansion is ignored.
Parsing result is valid.
```

```

vennela_07@VennelaPC:~/single_quote_parser$ ./quote_parser
Enter a command: echo ''

--- Parsing Result ---
Original Input : echo ''
Stored String  : echo

--- Validation ---
Single quotes handled successfully.
Literal content preserved.
Variable expansion is ignored.
Parsing result is valid.
vennela_07@VennelaPC:~/single_quote_parser$ ./quote_parser
Enter a command: echo 'Hello World'

Syntax Error: Unclosed single quote.

```

```

GNU nano 7.2 quote_parser.c
#include <stdio.h>
#include <string.h>

#define MAX 200

int main()
{
    char input[MAX];
    char output[MAX];
    int i = 0, j = 0;
    int inside_quote = 0;

    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty input.\n");
        return 0;
    }

    while (input[i] != '\0')
    {
        if (input[i] == '\'' )
        {
            inside_quote = !inside_quote;
            i++;
        }
        else
        {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

```

```

    if (inside_quote)
    {
        printf("\nSyntax Error: Unclosed single quote.\n");
        return 1;
    }

    printf("\n--- Parsing Result ---\n");
    printf("Original Input : %s\n", input);
    printf("Stored String : %s\n", output);

    printf("\n--- Validation ---\n");
    printf("Single quotes handled successfully.\n");
    printf("Literal content preserved.\n");
    printf("Variable expansion is ignored.\n");
    printf("Parsing result is valid.\n");

    return 0;
}

```

Task 2 : To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```

vennela_07@VennelaPC:~$ mkdir double_quote_parser
cd double_quote_parser
vennela_07@VennelaPC:~/double_quote_parser$ pwd
/home/vennela_07/double_quote_parser
vennela_07@VennelaPC:~/double_quote_parser$ nano double_quote_parser.c

vennela_07@VennelaPC:~/double_quote_parser$ gcc double_quote_parser.c -o double_quote_parser
vennela_07@VennelaPC:~/double_quote_parser$ export NAME="Vennela"
vennela_07@VennelaPC:~/double_quote_parser$ echo $NAME
Vennela
vennela_07@VennelaPC:~/double_quote_parser$ ./double_quote_parser
Enter command: echo "Hello World"

--- Parsing Result ---
Original Input : echo "Hello World"
Parsed Output : echo Hello World

--- Validation ---
Double quotes handled successfully.
Spaces inside quotes preserved.
Variable expansion performed.
Quoted command parsed successfully.
vennela_07@VennelaPC:~/double_quote_parser$ echo "Hello World"
Hello World

```

```

vennela_07@VennelaPC:~/double_quote_parser$ ./double_quote_parser
Enter command: echo "Hello $NAME"

--- Parsing Result ---
Original Input : echo "Hello $NAME"
Parsed Output  : echo Hello Vennela

--- Validation ---
Double quotes handled successfully.
Spaces inside quotes preserved.
Variable expansion performed.
Quoted command parsed successfully.
vennela_07@VennelaPC:~/double_quote_parser$ ./double_quote_parser
Enter command: echo "This is a long sentence"

--- Parsing Result ---
Original Input : echo "This is a long sentence"
Parsed Output  : echo This is a long sentence

--- Validation ---
Double quotes handled successfully.
Spaces inside quotes preserved.
Variable expansion performed.
Quoted command parsed successfully.

```

```

GNU nano 7.2 double_quote_parser.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 500

int main()
{
    char input[MAX];
    char output[MAX];
    char variable[100];
    int i = 0, j = 0;
    int inside_quotes = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        return 0;
    }

    while (input[i] != '\0')
    {
        /* Start or end double quotes */
        if (input[i] == '"')
        {
            inside_quotes = !inside_quotes;
            i++;
        }
    }
}

```

```

        while ((input[i] >= 'A' && input[i] <= 'Z') ||
               (input[i] >= 'a' && input[i] <= 'z') ||
               (input[i] >= '0' && input[i] <= '9') ||
               input[i] == '_')
        {
            variable[k++] = input[i++];
        }

        variable[k] = '\0';

        char *value = getenv(variable);

        if (value != NULL)
        {
            strcpy(&output[j], value);
            j += strlen(value);
        }
        else
        {
            printf("Warning: Variable %s not found.\n", variable);
        }
    }

    else
    {
        output[j++] = input[i++];
    }
}

output[j] = '\0';

/* Validate quotes */
if (inside_quotes)
{
    printf("\nSyntax Error: Unclosed double quote.\n");
    return 1;
}

printf("\n--- Parsing Result ---\n");
printf("Original Input : %s\n", input);
printf("Parsed Output  : %s\n", output);

printf("\n--- Validation ---\n");
printf("Double quotes handled successfully.\n");
printf("Spaces inside quotes preserved.\n");
printf("Variable expansion performed.\n");
printf("Quoted command parsed successfully.\n");

return 0;
}

```